<Company Name>

Arithmetic Expression Evaluator in C++
Software Requirements Specifications

Version <1.0>

[Note: The following template is provided for use with the Unified Process for EDUcation. Text enclosed in square brackets and displayed in blue italics (style=InfoBlue) is included to provide guidance to the author and should be deleted before publishing the document. A paragraph entered following this style will automatically be set to normal (style=Body Text).]

[To customize automatic fields in Microsoft Word (which display a gray background when selected), select File>Properties and replace the Title, Subject and Company fields with the appropriate information for this document. After closing the dialog, automatic fields may be updated throughout the document by selecting Edit>Select All (or Ctrl-A) and pressing F9, or simply click on the field and press F9. This must be done separately for Headers and Footers. Alt-F9 will toggle between displaying the field names and the field contents. See Word help for more information on working with fields.]

Revision History

Date	Version	Description	Author
14/3/2026	1.0	Completed the initial draft of Section 1 and part of Section 2. Also began work on Sections 4 and 5 and helped establish the structure of the SRS document.	RG
15/3/2026	1.1	Completed the remaining portions of Section 2 and reviewed the document for organization and consistency. Helped refine formatting and ensure the document followed by the SRS template structure.	Ayesha Akbar
15/3/2026	1.2	Completed Sections 3, 4, and 5 and reviewed the document for spelling and grammar. Also helped refine requirement descriptions and ensured consistency across the requirements listed in the document	Mahathi Sai

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 4
   - 1.5 Overview 4

2. Overall Description 5
   - 2.1 Product perspective 5
     - 2.1.1 System Interfaces 5
     - 2.1.2 User Interfaces 5
     - 2.1.3 Hardware Interfaces 5
     - 2.1.4 Software Interfaces 5
     - 2.1.5 Communication Interfaces 5
     - 2.1.6 Memory Constraints 5
     - 2.1.7 Operations 5
   - 2.2 Product functions 5
   - 2.3 User characteristics 5
   - 2.4 Constraints 5
   - 2.5 Assumptions and dependencies 5
   - 2.6 Requirements subsets 5

3. Specific Requirements 5
   - 3.1 Functionality 5
     - 3.1.1 <Functional Requirement One> 6
   - 3.2 Use-Case Specifications 6
   - 3.3 Supplementary Requirements 6

4. Classification of Functional Requirements 6

5. Appendices 6

Software Requirements Specifications

1. Introduction

This Software Requirements Specification (SRS) describes the requirements for the Arithmetic Expression Evaluator project developed for EECS 348 — Software Engineering. The document defines the functional and non-functional requirements of the system and provides a shared understanding of how the software should behave and what features it must support.
The introduction section provides an overview of the SRS and the project itself. It includes the purpose of the document, the scope of the system, definitions and terminology used throughout the document, references to related materials, and an outline of the remaining sections. Together, these elements establish the context for the requirements described in the rest of the specification.

1.1 Purpose

The purpose of this document is to define and communicate the software requirements for our Arithmetic Expression Evaluator project. As a team, we prepared this SRS to ensure we all share the same understanding of what the system needs to do before we begin designing and building it.

This document describes what the system must do, how it should behave, what constraints we are working under, and how we have prioritized features. It will serve as the foundation for our design document, test cases, and final implementation.

1.2 Scope

Our project is called AEEDevelopment. We are building an arithmetic expression evaluator in C++ that can take an expression entered by a user, parse it, and compute the correct result. The program needs to follow standard order of operations (PEMDAS), handle parentheses, support multiple arithmetic operators, and give the user clear output, whether that is a result or an error message.

The project description specifies a command-line interface, but our team has decided to extend the project by building a Qt-based graphical user interface with additional features like parse tree visualization and a token stream display. This SRS covers both the required core behavior and our team's extended scope.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
GUI	Graphical User Interface

PEMDAS	Order of operations: Parentheses, Exponents, Multiplication/Division, Addition/Subtraction
Tokenizer	The part of the program that breaks the input expression into individual components
Parser	The part of the program that organizes tokens into a structure the evaluator can process
Expression Tree	A tree data structure representing the arithmetic expression
Essential	A requirement that must be implemented for the project to meet its goals
Desirable	A requirement we plan to implement but is not strictly required
Optional	A feature we will implement only if we have time

1.4 References

EECS 348 Term Project Description — Arithmetic Expression Evaluator in C++

UPEDU Software Requirements Specification Template

AEEDevelopment GitHub README

Team Programming Guidelines Document

Team Project Plan Document

1.5 Overview

Section 2 gives background on our product, including what it does, who will use it, and what constraints we are working with. Section 3 lists all of our functional and non-functional requirements in detail. Section 4 classifies those requirements by priority. The appendices include examples of valid and invalid expressions taken directly from the project description.

2. Overall Description

The overall description provides background information about the Arithmetic Expression Evaluator system and the environment in which it operates. This section explains the general characteristics of the product, including its purpose, major functions, intended users, constraints, and assumptions that influence development. Rather than listing specific requirements, this section describes the broader context of the system so that the detailed functional and non-functional requirements presented in Section 3 can be more easily understood. It also outlines how the system fits within the scope of the EECS 348 course project and the design decisions made by the development team.

2.1 Product perspective

Our arithmetic expression evaluator is a standalone application that we are building from scratch as part of this software engineering course. According to the project description, it is conceptually a parser component for a larger compiler for a programming language called Language L. For the purposes of this course, we are developing the expression parser and evaluator portion.

We organized the system into clear modules: a tokenizer, parser, evaluator, expression tree, error handler, and user interface. Keeping these components separate makes the code easier to test, maintain, and extend.

2.1.1 System Interfaces

The system interacts with the user through the front-end interface, and internally the tokenizer, parser, evaluator, and error handler communicate through defined interfaces. Each module has a clear responsibility and passes structured data to the next stage of processing.

2.1.2 User Interfaces

We are building a Qt-based graphical user interface for our application. The interface will include an expression input field, a results panel, and optional panels for displaying the parse tree, token stream, and step-by-step evaluation. We also plan to add a dark mode toggle and expression history as desirable features.

2.1.3 Hardware Interfaces

The system does not require any specialized hardware interfaces. The application will run on a standard personal computer and will rely only on typical input and output devices such as a keyboard, mouse, and display. Users will interact with the system through the graphical user interface to enter arithmetic expressions and view results. No additional hardware components or external devices are required for the system to function.

2.1.4 Software Interfaces

Our project is written in C++ using Qt 6.10 for the graphical user interface layer, CMake for build management, and Git/GitHub for version control. The system is compiled using Clang 14 or newer, and Qt Test is used for implementing and running unit tests to verify the functionality of different system components.

2.1.5 Communication Interfaces

The system does not require any external communication interfaces. The application operates as a standalone program on a local machine and does not rely on network communication or external services. All processing, including input handling, parsing, evaluation, and result of display, occurs within the application itself without the need for data exchange with other systems.

2.1.6 Memory Constraints

There is no specific memory constraints listed in the project description. Our application is designed for normal classroom-sized expressions, and we will ensure that the program does not use unnecessary memory during evaluation.

2.2 Product functions

At a high level, our system will:

- Let users enter arithmetic expressions through the interface
- Break the input into tokens (numbers, operators, parentheses)
- Parse those tokens into a structured expression tree
- Evaluate the expression correctly using precedence and associativity rules
- Support all required operators including exponentiation
- Handle unary + and unary − operators
- Allow nested parentheses to group sub-expressions
- Display the correct result or a clear error message
- Optionally show the parse tree, token stream, and evaluation steps

2.3 User characteristics

Our main users are students and instructors in a software engineering course. They are familiar with standard arithmetic but do not necessarily understand how parsers or compilers work internally. For that reason, we want the interface to be simple and intuitive, and the error messages to be clear enough for users to understand without needing knowledge of the underlying code or system design.

2.4 Constraints

- The system must be written in C++.
- We must follow object-oriented programming principles.
- The system must support all operators described in the project requirements.
- Operator precedence and association must be handled correctly.
- Error handling must be robust, and the program should never crash when given invalid input.
- All codes must include appropriate documentation.
- The project must include unit tests.
- All project materials and source code must be version-controlled using GitHub.

2.5 Assumptions and dependencies

We are assuming that the initial version of the program will accept only integer input. The project description indicates that floating-point support may be requested later as a change request, so we are currently treating it as a desirable feature rather than a core requirement. We also assume that the design document, test cases, and user manual will remain aligned with this SRS throughout the rest of the development process.

3. Specific Requirements

3.1 Functionality

3.2 Functional Requriment-1 — Expression Input

The system shall allow the user to enter an arithmetic expression through the interface.

3.3 Functional Requriment-2 — Tokenization

The system shall break the input into tokens, including numbers, operators, and parentheses, before parsing.

3.4 Functional Requriment-3 — Binary Operator Support

The system shall support the following binary operators: + (addition), - (subtraction), * (multiplication), / (division), % (modulo), and ** (exponentiation).

3.5 Functional Requriment-4 — Unary Operator Support

The system shall support unary + and unary - applied to operands or grouped sub-expressions.

3.6 Functional Requriment-5 — Parentheses

The system shall support parentheses for grouping, including nested and extraneous parentheses.

3.7 Functional Requriment-6 — Operator Precedence

The system shall evaluate expressions following PEMDAS-based precedence rules.

3.8 Functional Requriment-7 — Operator Associativity

The system shall evaluate +, -, *, /, and % from left to right. The system shall evaluate ** from right to left.

3.9 Functional Requriment-8 — Numeric Constants

The system shall recognize and evaluate integer numeric constants in the expression.

3.10 Functional Requriment-9 — Result Display

The system shall display the computed result when a valid expression is entered.

3.11 Functional Requriment-10 — Invalid Character Detection

The system shall detect any invalid or unsupported characters and report them as errors.

3.12 Functional Requriment-11 — Syntax Error Detection

The system shall detect malformed expressions, including unmatched parentheses, missing operands, invalid operator sequences, and missing operators between adjacent values.

3.13 Functional Requriment-12 — Division by Zero Handling

The system shall detect division by zero and display an appropriate error message instead of attempting to evaluate.

3.14 Functional Requriment-13 — Error Messaging

The system shall display error messages that are clear and specific enough to help the user understand what went wrong.

3.15 Functional Requriment-14 — Internal Expression Representation

The system shall build an internal representation of the expression (such as an expression tree or stack-based structure) to support correct evaluation.

3.16 Functional Requriment-15 — User Interface

The system shall provide a user-friendly interface for submitting expressions and viewing results or errors.

3.17 Functional Requriment-16 — Unit Testing

Our team shall write unit tests that cover tokenizer, parser, evaluator, and error-handling logic.

3.18 Functional Requriment-17 — Parse Tree Visualization

The system should display a visual parse tree for successfully evaluated expressions.

3.19 Functional Requriment-18 — Step-by-Step Evaluation

The system should break down and show evaluation steps for valid expressions.

3.20 Functional Requriment-19 — Token Stream Display

The system should display the token stream generated from the user's input.

3.21 Functional Requriment-20 — Floating-Point Support

The system should support floating-point numbers in a future release or as an extended feature.

3.22 Functional Requriment-21 — Expression History

The system may log and display previously evaluated expressions during a session.

3.23 Functional Requriment-22 — Dark Mode

The system may offer a dark mode display setting.

3.24 Functional Requriment-23 — Bracket Grouping

The system may support square brackets [ ] as additional grouping symbols.

3.25 Functional Requriment-24 — Algebraic Variables

The system may support algebraic variables in future versions if scope is expanded.

3.26 Use-Case Specifications

Use Case 1 — Evaluate a Valid Expression

Actor: User

Precondition: The application is running

Flow:

1) User types a valid arithmetic expression

2)User submits it

3)System tokenizes the input

4)System parses the tokens

5)System evaluates the expression

6)System displays the result

Postcondition: The correct result is shown.

3.27 Use Case 2 — Reject an Invalid Expression

Actor: User

Precondition: The application is running

Flow:

1. User enters an invalid expression
2. User submits it
3. System detects the error during tokenization, parsing, or evaluation
4. System displays a descriptive error message

Postcondition: The user sees a helpful error; no false result is shown.

3.28 Use Case 3 — View Visualization Details

Actor: User

Precondition: Visualization features are available; expression is valid

Flow:

1. User evaluates a valid expression
2. User navigates to the visualization panel
3. System displays the parse tree, token stream, or evaluation steps

Postcondition: User can inspect internal structure of their expression.

3.29 Supplementary Requirements

3.30 Performance

The system should evaluate expressions with no noticeable delay under normal use conditions.

3.31 Reliability

The system shall not crash under any invalid input. All errors should be caught and reported gracefully.

3.32 Usability

The interface should be simple and clean. Error messages must be written in plain language.

3.33 Maintainability

We will follow object-oriented design and keep components modular, so the code is easy to update as the project progresses.

3.34 Testability

Every functional requirement should be verifiable through unit tests or input/output testing.

3.35 Documentation

The project will include a README, SRS, design document, test plan, and user manual as required by the course.

3.36 Portability

The project should build and run on Windows, macOS, and Linux using our team's documented toolchain.

4. Classification of Functional Requirements

Requirement	Description	Priority
Functional Requirement-1	Expression Input	Essential

Functional Requirement -2	Tokenization	Essential
Functional Requirement -3	Binary Operator Support	Essential
Functional Requirement -4	Unary Operator Support	Essential
Functional Requirement -5	Parentheses	Essential
Functional Requirement -6	Operator Precedence	Essential
Functional Requirement -7	Operator Associativity	Essential
Functional Requirement -8	Numeric Constants	Essential
Functional Requirement -9	Result Display	Essential
Functional Requirement -10	Invalid Character Detection	Essential
Functional Requirement -11	Syntax Error Detection	Essential
Functional Requirement -12	Division by Zero Handling	Essential
Functional Requirement -13	Error Messaging	Essential
Functional Requirement -14	Internal Expression Representation	Essential
Functional Requirement -15	User Interface	Essential
Functional Requirement -16	Unit Testing	Essential

Functional Requirement -17	Parse Tree Visualization	Desirable
Functional Requirement -18	Step-by-Step Evaluation	Desirable
Functional Requirement -19	Token Stream Display	Desirable
Functional Requirement -20	Floating-Point Support	Desirable
Functional Requirement -21	Expression History	Optional
Functional Requirement -22	Dark Mode	Optional
Functional Requirement -23	Bracket Grouping	Optional
Functional Requirement -24	Algebraic Variables	Optional

Appendices

4.1 Appendix A — Valid Expression Examples

Expression	Expected Result
3 + 4	7
8 - (5 - 2)	5
10 * 2 / 5	4
2 ** 3	8
4 * (3 + 2) % 7 - 1	5
(((2 + 3))) + (((1 + 2)))	8
+(-2) * (-3) - ((-4) / (+5))	6.8
2 ** 3 ** 2	512

Appendix B — Invalid Expression Examples

Expression	Reason
2 * (4 + 3 - 1	Unmatched parenthesis
* 5 + 2	Operator missing left operand

4 / 0	Division by zero
5 (2 + 3)	Missing operator
7 & 3	Invalid character
((2 -) 1 + 3)	Invalid operator sequence

4.2 Appendix C — Requirement Traceability

All Essential requirements trace directly to the project description provided by Professor Saiedian. Desirable requirements are based on the project description mention of future floating-point support as a likely change request, and our team's extended feature plans described in the repository README and programming guidelines. Optional requirements reflect additional stretch-goal features our team identified during planning.